

binsdfc

0.1

Generated by Doxygen 1.5.6

Wed Jul 21 12:09:32 2010

Contents

1	Main Page	1
2	Class Index	5
2.1	Class Hierarchy	5
3	Class Index	7
3.1	Class List	7
4	Class Documentation	9
4.1	aboveNoise Class Reference	9
4.1.1	Detailed Description	9
4.1.2	Member Function Documentation	9
4.1.2.1	operator()	9
4.2	binboundary_expectation Class Reference	11
4.2.1	Detailed Description	11
4.2.2	Member Function Documentation	11
4.2.2.1	operator()	11
4.3	binboundary_expectation2 Class Reference	12
4.3.1	Detailed Description	12
4.3.2	Member Function Documentation	12
4.3.2.1	operator()	12
4.4	binboundary_probability Class Reference	13
4.4.1	Detailed Description	13
4.4.2	Member Function Documentation	13
4.4.2.1	operator()	13
4.5	evidenceComputer< AVGFUNC > Class Template Reference	14
4.5.1	Detailed Description	14
4.5.2	Member Function Documentation	15
4.5.2.1	compute	15

4.5.2.2	getEvidence	15
4.5.2.3	getEvidenceSum	15
4.5.2.4	setFunctorParams	15
4.5.2.5	test	15
4.5.2.6	computeEvidences	15
4.5.3	Member Data Documentation	15
4.5.3.1	mAF	15
4.5.3.2	mEvidences	16
4.5.3.3	mSubEvidences	16
4.6	evifunc Class Reference	17
4.6.1	Detailed Description	18
4.6.2	Member Function Documentation	18
4.6.2.1	init	18
4.6.2.2	operator()	18
4.6.3	Member Data Documentation	18
4.6.3.1	mBin	18
4.6.3.2	mT	18
4.6.3.3	mProb	19
4.6.3.4	mIP0	19
4.6.3.5	mPUB	19
4.7	findPriorExp Class Reference	20
4.7.1	Detailed Description	20
4.7.2	Member Function Documentation	20
4.7.2.1	evaluate	20
4.7.2.2	checkConstraints	20
4.8	inhib_latency Class Reference	22
4.8.1	Detailed Description	22
4.9	inhib_latsum Class Reference	23
4.9.1	Detailed Description	23
4.10	latency Class Reference	24
4.10.1	Detailed Description	24
4.10.2	Member Function Documentation	24
4.10.2.1	operator()	24
4.11	latsum Class Reference	25
4.11.1	Detailed Description	25
4.11.2	Member Function Documentation	25

4.11.2.1	operator()	25
4.12	p_firing_rate Class Reference	26
4.12.1	Detailed Description	26
4.12.2	Member Function Documentation	26
4.12.2.1	init	26
4.12.2.2	operator()	26
4.13	p_firing_rate_and_latency Class Reference	28
4.13.1	Detailed Description	28
4.13.2	Member Function Documentation	28
4.13.2.1	init	28
4.13.2.2	operator()	28
4.14	SDF Class Reference	30
4.14.1	Detailed Description	30
4.14.2	Member Function Documentation	30
4.14.2.1	operator()	30
4.15	SDF2 Class Reference	31
4.15.1	Detailed Description	31
4.15.2	Member Function Documentation	31
4.15.2.1	operator()	31
4.16	SDF2p Class Reference	32
4.16.1	Detailed Description	32
4.16.2	Member Function Documentation	32
4.16.2.1	operator()	32
4.17	SDFp Class Reference	33
4.17.1	Detailed Description	33
4.17.2	Member Function Documentation	33
4.17.2.1	operator()	33
4.18	simplexMinimiser Class Reference	34
4.18.1	Detailed Description	35
4.18.2	Member Function Documentation	35
4.18.2.1	init	35
4.18.2.2	init	35
4.18.2.3	evaluate	35
4.18.2.4	checkConstraints	36
4.18.2.5	findMinimum	36
4.18.2.6	hasConverged	36

4.18.2.7	getNumSteps	36
4.18.2.8	test	36
4.18.2.9	findHiLo	36
4.18.2.10	extrapolate	36
4.18.2.11	shrink	36
4.18.2.12	iterate	37
4.18.2.13	findMaxStretch	37
4.18.2.14	computeTestpoint	37
4.18.3	Member Data Documentation	37
4.18.3.1	mSimplex	37
4.18.3.2	mTestpoint	37
4.18.3.3	mHigh	37
4.18.3.4	mTol	37
4.18.3.5	mSteps	37
4.19	spikeCounter Class Reference	38
4.19.1	Detailed Description	39
4.19.2	Member Function Documentation	39
4.19.2.1	setMaxM	39
4.19.2.2	setInterval	39
4.19.2.3	setIntervalPrior	39
4.19.2.4	clearData	39
4.19.2.5	addData	39
4.19.2.6	getSpikes	40
4.19.2.7	getGaps	40
4.19.2.8	test	40
4.19.2.9	allocArrays	40
4.19.2.10	precomputeSubIntervals	40
4.19.2.11	computePrior	40
4.19.3	Member Data Documentation	40
4.19.3.1	mIntervalEvidences	40
4.19.3.2	mIntervalCounts	40
4.19.3.3	mPriors	41
4.19.3.4	mPrior1	41
4.19.3.5	mIntervStart	41
4.19.3.6	mMMax	41
4.19.3.7	mSpikeTrain	41

4.19.3.8	mCurSp	41
4.19.3.9	mAutoCorr	41
4.19.3.10	mbDataChanged	41
4.19.3.11	mPUB	41
4.20	spikeDensityFunction Class Reference	42
4.20.1	Detailed Description	43
4.20.2	Member Function Documentation	44
4.20.2.1	setMaxM	44
4.20.2.2	setIntervalPriors	44
4.20.2.3	setTimeInterval	44
4.20.2.4	clearData	44
4.20.2.5	addData	44
4.20.2.6	findMInterval	44
4.20.2.7	setMInterval	44
4.20.2.8	getMProb	44
4.20.2.9	getSDF	44
4.20.2.10	getSDFStdDev	45
4.20.2.11	getBBProb	45
4.20.2.12	getBBExpectation	45
4.20.2.13	getBBStdDev	45
4.20.2.14	getPAbNoise	45
4.20.2.15	getLatencyProb	45
4.20.2.16	getLatencyProbs	46
4.20.2.17	getBABProb	46
4.20.2.18	getNOSIG	46
4.20.2.19	getSeparationProb	46
4.20.2.20	findBestSeparator	46
4.20.2.21	getFiringRateGivenLatencyProb	47
4.20.2.22	getFiringRateProb	47
4.20.2.23	getTotalEvidence	47
4.20.2.24	getIntervalEvidenceSum	47
4.20.2.25	getEvidence	47
4.20.2.26	getInhibitoryLatencyProb	47
4.20.2.27	getInhibitorySeparationProb	48
4.20.2.28	findBestInhibitorySeparator	48
4.20.2.29	goldenSectionSearch	48

4.20.2.30	computeEvidences	48
4.20.3	Member Data Documentation	48
4.20.3.1	mMaxM	48
4.20.3.2	mEV	48
4.20.3.3	mM1	49
4.20.3.4	mP0	49
4.20.3.5	mIntervStart	49
4.20.3.6	mbDataChanged	49
4.20.3.7	mbMIntervalChanged	49
4.20.3.8	mEvSum	49
4.20.3.9	mEvIntervSum	49
4.20.3.10	mWndstartLastTime	49
4.20.3.11	mTex1	49

Chapter 1

Main Page

Author:

Dominik M Endres

binsdfc is a command line implementation of the algorithm described in [Endres, Oram, Schindelin, Foldiak \(2007\): Bayesian binning beats approximate alternatives: estimating peri-stimulus time histograms](#). It computes spike density functions ([SDF](#)) or peri-stimulus time histograms (PSTH). Given that it performs exact Bayesian averaging, the result is somewhere in the middle between the two: while the underlying model is comprised of bins like a PSTH, the averaging process gives rise to a more "continuous" prediction like a [SDF](#). If you haven't read the paper, do it now.

binsdfc accepts input from stdin or a supplied file, and computes the expected [SDF](#), its variance, latencies (as described in [Endres, Schindelin, Foldiak, Oram \(2010\): Modelling spike trains and extracting response latency with Bayesian binning](#)), and various other averages. The central iteration is implemented in the template class [evidenceComputer](#), the functions to be averaged implement the interface [evifunc](#). The way these two interact follow the formalism developed in [Endres and Foldiak \(2005\), Bayesian bin distribution inference and mutual information](#).

Usage: binsdf [options] filename.

Compute spike density functions. Input either from file or stdin (if filename='-').

Input file format: 1 trial per line, spike times aligned to stimulus onset, line format:

<stimulus name> <spike time 1> <spike time 2> <spike time N>"<<endl

--help: this text

--start,-s: start time index for sdf calculations

--end,-e: end time index

--efire,-f: prior exponent of the Beta distribution of each bin for firing

--egap,-g: prior exponent of the Beta distribution of each bin for not firing

--max-num-bins,-m: maximal number of bin boundaries for sdf inside the time interval

--sdfvar,-v: calculate variance of spike density function.

--m-posterior,-o: calculate number-of-bin-boundaries posterior instead of sdf

--p-post-lbound,-l: lower bound of posterior probability that chosen number-of-bin-boundaries

--bin-boundaries,-b <number of bin boundaries>: compute expectations and std.devs of bin bound

--bin-boundary-posterior,-p <number of bin boundaries>: compute posterior distributions of bin

--find-prior-exponents,-P: find most probable prior exponents

--find-signal-separation-level,-y: find best signal separation level

--find-latency,-L <signal separation level>: find [latency](#) based on <signal separation level>

--find-inhibitory-signal-separation-level,-i: find best inhibitory signal separation level"

--find-inhibitory-latency,-I <inhibitory signal sep. level>: find inhibitory [latency](#) based on

```
--latency-start,-S: start time index for latency calculations
--latency-end,-E: end time index for latency calculations
--no-sdf, -n: don't compute sdf
--marginal-likelihood, -M: return list with  $\log(P(\text{data}|M))$ , for M between 0 and --max-num-bins
```

Very short and incomplete tutorial

Building the sources: we use the cmake build system:

```
cd binsdfc
cmake .
make
```

and feel free to add more optimisation options to CMakeLists.txt .

If you would like to rebuild this documentation, install doxygen and call it from the source directory.

The `binsdfc/src/` directory contains a python script called **testdata.py** which create some data in the required format and print them to stdout. Just pipe them into binsdfc, like

```
./testdata.py | ./binsdfc -s -100 -e 500
```

and you should get the [SDF](#) to stdout, in the interval from -100ms pre-stimulus (or whatever unit you use) to 500ms post-stimulus. Now let's see how certain we can be of the values we just got by adding the -v option for the standard deviation of the [SDF](#):

```
./testdata.py | ./binsdfc -s -100 -e 500 -v
```

You might have noticed that the first line of the output (actually sent to stderr) says something like

```
Including models with  $3 \leq M \leq 3$  at probability 0.292397
```

This means that the most probable M (number of bin boundaries) is 3, and the posterior probability $P(M|D)$ is about 0.29. The prior over M is assumed to be uniform, and within 0..10. If you'd like to consider a larger range of bin numbers, use the `-max-num-bins` option.

You might feel that a probability of 0.29 that the model ($M=3$) is the correct one (within the model class, as usual in Bayesian analyses) is not high enough. So let's raise the lower bound on the M posterior probability by using -l:

```
./testdata.py | ./binsdfc -s -100 -e 500 -v -l 0.9
```

Now the first line of the output will look something like

```
Including models with  $3 \leq M \leq 5$  at probability 0.932465
```

i.e. to be more than 90% sure that you include the correct model, you have to include models with $3 \leq 5 \leq M$ in all subsequent calculations. If you compare the [SDF](#) computed with this setting with the previous one, you should notice slight differences.

You can also tweak the hyperparameters of the Beta prior on the firing rate probabilities, using the options `-f` for the exponent of the firing probability and `-g` for the exponent of the not-firing probability. The default setting is `-f 1 -g 32`, which reproduces the average firing rate of an STSa neuron under experimental condition. Alternatively, you can try to optimise these hyperparameters with the `-P` options (caution: this feature is still experimental, and the optimisation might get stuck. If you end up with a hyperparameter less than 0.001 or more than 300.0, something's gone wrong. Please let me know.):

```
./testdata.py | ./binsdfc -s -100 -e 500 -v -l 0.9 -P
```

which should give you an output like

```
Best values for efire, egap = 0.170484, 2.07251, firing rate 0.0760071+-0.147159
```

at the beginning. This minimisation might take a while, but if nothing happens for a minute, it might have gotten stuck.

Now let's find the [latency](#). First, we need to determine the signal separation level, i.e. that firing probability below which we'll classify the cell as "not responding":

```
./testdata.py | ./binsdfc -s -100 -e 500 -l 0.9 -P -y
```

(of course you could have supplied the previously found values for the beta hyperparameters with `-f`, `-g` instead of running the optimisation again). You should get an output like:

```
Best signal separation level: 0.0441759 at probability 0.999398
```

This means that at the signal separation level of 0.0441759, the probability for the existence of a [latency](#) (i.e. of a signal) is 0.999398 – we can be reasonably confident that this cell is responding. Let's see where exactly by computing the [latency](#) posterior distribution with the `-L` option:

```
./testdata.py | ./binsdfc -s -100 -e 500 -l 0.9 -P -L 0.0441759
```

You will hopefully see that the [latency](#) posterior has a sharp peak at 80ms, which is exactly where the test data are simulating a response.

Chapter 2

Class Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

evifunc	17
aboveNoise	9
binboundary_expectation	11
binboundary_expectation2	12
binboundary_probability	13
inhib_latency	22
inhib_latsum	23
latency	24
latsum	25
p_firing_rate	26
p_firing_rate_and_latency	28
SDF	30
SDF2	31
SDF2p	32
SDFp	33
simplexMinimiser	34
findPriorExp	20
spikeCounter	38
evidenceComputer< AVGFUNC >	14
spikeDensityFunction	42

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

aboveNoise (Compute probability that firing probability is above aboveNoise::mProb , which might e.g. represent a noise level, or a background firing rate, or some such)	9
binboundary_expectation (Functor for computing the expected position of a bin boundary) . . .	11
binboundary_expectation2 (Functor for bin boundary ² expectation, to compute posterior variance of bin boundary positions)	12
binboundary_probability (Functor for probability that a upper bin boundary is at a given time index)	13
evidenceComputer< AVGFUNC > (The template for the central iteration of the algorithm, as described in Endres, Oram, Schindelin, Foldiak (2007): Bayesian binning beats approximate alternatives: estimating peri-stimulus time histograms)	14
evifunc (Evidence calculation, i.e. when this functor is used, the algorithm as described in the paper is implemented)	17
findPriorExp	20
inhib_latency (Probability that inhibitory latency is at inhib_latency::mT , i.e. that the firing probability was above inhib_latency::mProb (this is the "inhibitory signal separation level") prior to inhib_latency::mT , and is below latency::mProb for at least 1 bin after inhib_latency::mT)	22
inhib_latsum	23
latency (Probability that latency is at latency::mT , i.e. that the firing probability was below latency::mProb (this is the "signal separation level" or noise level) prior to latency::mT , and is above latency::mProb for at least 1 bin after latency::mT)	24
latsum (To compute the probability that there is a signal somewhere in the spiketrain, the averages of the latency functor need to be summed over all time indexes in the interval of interest)	25
p_firing_rate (Functor for the sum of p_firing_rate_and_latency across all time indexes in the interval of interest)	26
p_firing_rate_and_latency (Functor for probability density that (firing probability==log(p_firing_rate_and_latency::mProb))== p_firing_rate_and_latency::mLP at p_firing_rate_and_latency::mT and firing probability prior to that was below p_firing_rate_and_latency::mProb)	28
SDF (Functor for spike density function)	30
SDF2 (Functor for spike density function ² (to compute variance of SDF))	31

SDF2p (Functor for spike density function ² (to compute variance of SDF) with variable upper bound on pfire)	32
SDFp (Functor for spike density function with variable upper bound on pfire)	33
simplexMinimiser (Perform minimum search in N-dim by simplex method (see Numerical Recipes, chapter 10))	34
spikeCounter (Counts spike trains and stores results)	38
spikeDensityFunction (Compute bin spike density function and related quantities, e.g. latency , window end, signal probability)	42

Chapter 4

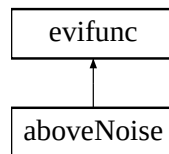
Class Documentation

4.1 aboveNoise Class Reference

compute probability that firing probability is above [aboveNoise::mProb](#), which might e.g. represent a noise level, or a background firing rate, or some such.

```
#include <evidencecomputer.h>
```

Inheritance diagram for aboveNoise::



Public Member Functions

- double [operator\(\)](#) (const int &lb, const int &ub, const int &binno, pair< int, int > &numpts, const double &evcont)

4.1.1 Detailed Description

compute probability that firing probability is above [aboveNoise::mProb](#), which might e.g. represent a noise level, or a background firing rate, or some such.

4.1.2 Member Function Documentation

- 4.1.2.1** double aboveNoise::operator() (const int & *lb*, const int & *ub*, const int & *binno*, pair< int, int > & *numpts*, const double & *evcont*) [inline]

set [aboveNoise::mProb](#) to the noise level before iterating, and [aboveNoise::mIP0](#), aboveNoise::mIP1 to the exponents of the beta prior.

The documentation for this class was generated from the following file:

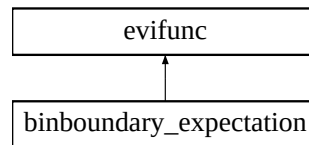
- `evidencecomputer.h`

4.2 binboundary_expectation Class Reference

functor for computing the expected position of a bin boundary

```
#include <evidencecomputer.h>
```

Inheritance diagram for binboundary_expectation::



Public Member Functions

- double [operator\(\)](#) (const int &lb, const int &ub, const int &binno, const pair< int, int > &numpts, const double &evcont)

4.2.1 Detailed Description

functor for computing the expected position of a bin boundary

4.2.2 Member Function Documentation

4.2.2.1 double binboundary_expectation::operator() (const int &lb, const int &ub, const int &binno, const pair< int, int > &numpts, const double &evcont) [inline]

before starting the iteration, set [binboundary_expectation::mBin](#) to the number of the bin whose expected upper bound you want to compute

Reimplemented from [evifunc](#).

The documentation for this class was generated from the following file:

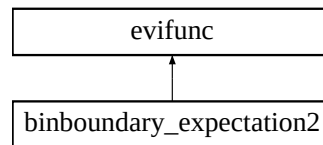
- evidencecomputer.h

4.3 binboundary_expectation2 Class Reference

functor for bin boundary² expectation, to compute posterior variance of bin boundary positions

```
#include <evidencecomputer.h>
```

Inheritance diagram for binboundary_expectation2::



Public Member Functions

- double [operator\(\)](#) (const int &lb, const int &ub, const int &binno, const pair< int, int > &numpts, const double &evcont)

4.3.1 Detailed Description

functor for bin boundary² expectation, to compute posterior variance of bin boundary positions

4.3.2 Member Function Documentation

4.3.2.1 double binboundary_expectation2::operator() (const int & *lb*, const int & *ub*, const int & *binno*, const pair< int, int > & *numpts*, const double & *evcont*) [inline]

before starting the iteration, set [binboundary_expectation::mBin](#) to the number of the bin whose expected upper bound you want to compute

Reimplemented from [evifunc](#).

The documentation for this class was generated from the following file:

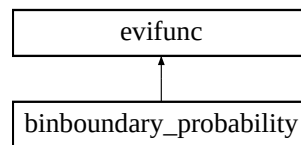
- evidencecomputer.h

4.4 binboundary_probability Class Reference

functor for probability that a upper bin boundary is at a given time index

```
#include <evidencecomputer.h>
```

Inheritance diagram for binboundary_probability::



Public Member Functions

- double [operator\(\)](#) (const int &lb, const int &ub, const int &binno, const pair< int, int > &numpts, const double &evcont)

4.4.1 Detailed Description

functor for probability that a upper bin boundary is at a given time index

4.4.2 Member Function Documentation

4.4.2.1 double binboundary_probability::operator() (const int & *lb*, const int & *ub*, const int & *binno*, const pair< int, int > & *numpts*, const double & *evcont*) [inline]

before starting the iteration, set [binboundary_probability::mBin](#) to the number of the bin whose probability of having its upper bound at [binboundary_probability::mT](#) you want to compute.

Reimplemented from [evifunc](#).

The documentation for this class was generated from the following file:

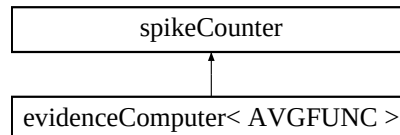
- evidencecomputer.h

4.5 evidenceComputer< AVGFUNC > Class Template Reference

The template for the central iteration of the algorithm, as described in [Endres, Oram, Schindelin, Foldiak \(2007\): Bayesian binning beats approximate alternatives: estimating peri-stimulus time histograms](#).

```
#include <evidencecomputer.h>
```

Inheritance diagram for evidenceComputer< AVGFUNC >::



Public Member Functions

- bool [compute](#) ()
- double [getEvidence](#) (int m)
- double [getEvidenceSum](#) (int m1, int m2)
- void [setFuncutorParams](#) (int bin1, int bin2, int t1, int t2, double prob, double prob2, int tex1=INT_MAX, int tex2=-INT_MAX)
- void [test](#) ()

Protected Member Functions

- void [computeEvidences](#) ()

Protected Attributes

- AVGFUNC [mAF](#)
- vector< double > [mEvidences](#)
- vector< double > [mSubEvidences](#)

4.5.1 Detailed Description

template<class AVGFUNC> class evidenceComputer< AVGFUNC >

The template for the central iteration of the algorithm, as described in [Endres, Oram, Schindelin, Foldiak \(2007\): Bayesian binning beats approximate alternatives: estimating peri-stimulus time histograms](#).

The template parameter is the function to be iterated over, which is in the innermost loop. Hence, using a compile-time resolvable parameter should speed up the averaging process. All computations in this object are done logarithmically, thus the averaging functor has to return the log of the values to be averaged.

Author:

Dominik Endres

4.5.2 Member Function Documentation

4.5.2.1 `template<class AVGFUNC> bool evidenceComputer< AVGFUNC >::compute ()`
`[inline]`

do precomputation, compute priors and compute evidences.

Returns:

true on success.

4.5.2.2 `template<class AVGFUNC> double evidenceComputer< AVGFUNC >::getEvidence (int`
`m) [inline]`

Returns:

log evidence for a model with m bin boundaries

4.5.2.3 `template<class AVGFUNC> double evidenceComputer< AVGFUNC >::getEvidenceSum`
`(int m1, int m2) [inline]`

Returns:

log sum of evidences for models with m1<=m<=m2

4.5.2.4 `template<class AVGFUNC> void evidenceComputer< AVGFUNC >::setFunctorParams`
`(int bin1, int bin2, int t1, int t2, double prob, double prob2, int tex1 = INT_MAX, int tex2`
`= -INT_MAX) [inline]`

set parameters of the averaging functor. Exact meanings will depend on which functor is used.

4.5.2.5 `template<class AVGFUNC> void evidenceComputer< AVGFUNC >::test () [inline]`

test this object

4.5.2.6 `template<class AVGFUNC> void evidenceComputer< AVGFUNC >::computeEvidences`
`() [inline, protected]`

compute Evidences for models with up to mMMax bin boundaries

4.5.3 Member Data Documentation

4.5.3.1 `template<class AVGFUNC> AVGFUNC evidenceComputer< AVGFUNC >::mAF`
`[protected]`

the averaging functor

4.5.3.2 `template<class AVGFUNC> vector<double> evidenceComputer< AVGFUNC >::mEvidences` [protected]

Evidences/averages for models with a given number of bins. The array E[] in the paper.

4.5.3.3 `template<class AVGFUNC> vector<double> evidenceComputer< AVGFUNC >::mSubEvidences` [protected]

sub-evidence array for the calculation. for details, see paper. The array subE[] in the paper.

The documentation for this class was generated from the following files:

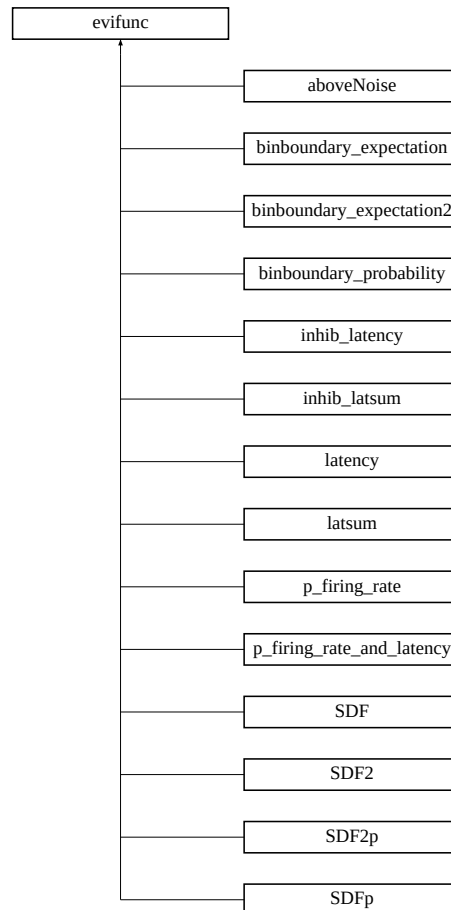
- `evidencecomputer.h`
- `evidencecomputer.cpp`

4.6 evifunc Class Reference

Evidence calculation, i.e. when this functor is used, the algorithm as described in the paper is implemented.

```
#include <evidencecomputer.h>
```

Inheritance diagram for evifunc::



Public Member Functions

- void [init](#) ()
- double [operator](#)() (const int &lb, const int &ub, const int &binno, const pair< int, int > &numpts, const double &evcont)

Public Attributes

- int [mBin](#)
- int [mBin2](#)
- int [mT](#)
- int [mT2](#)
- int [mTex1](#)
- int [mTex2](#)

- double [mProb](#)
- double [mLP](#)
- double [mL1mP](#)
- double [mProb2](#)
- double [mIP0](#)
- double [mIP1](#)
- double [mPUB](#)

4.6.1 Detailed Description

Evidence calculation, i.e. when this functor is used, the algorithm as described in the paper is implemented.

4.6.2 Member Function Documentation

4.6.2.1 void `evifunc::init()` [inline]

Possibly necessary initialisations. Call before starting to average.

Reimplemented in [p_firing_rate_and_latency](#), and [p_firing_rate](#).

4.6.2.2 double `evifunc::operator()(const int &lb, const int &ub, const int &binno, const pair<int, int> &numpts, const double &evcont)` [inline]

Parameters:

lb,: lower bound (exclusive!) of current bin

ub,: upper bound (inclusive!) of current bin

binno,: current bin number

numpts,: <spikes,gaps> observed in current bin.

evcont,: evidence contribution (getIEC() in the paper) of the current bin.

Returns:

log function value to be averaged. Here: just the evidence contribution of this interval.

Reimplemented in [binboundary_expectation](#), [binboundary_expectation2](#), [binboundary_probability](#), [SDF](#), [SDFp](#), [SDF2](#), [SDF2p](#), [p_firing_rate_and_latency](#), [p_firing_rate](#), and [latency](#).

4.6.3 Member Data Documentation

4.6.3.1 int `evifunc::mBin`

bin numbers

4.6.3.2 int `evifunc::mT`

time indexes

4.6.3.3 double evifunc::mProb

firing probabilities and their logarithms

4.6.3.4 double evifunc::mIP0

prior exponents of the beta distributions in the current interval

4.6.3.5 double evifunc::mPUB

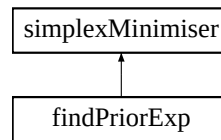
upper bound on firing probability

The documentation for this class was generated from the following file:

- evidencecomputer.h

4.7 findPriorExp Class Reference

Inheritance diagram for findPriorExp::



Public Member Functions

- virtual double [evaluate](#) (tPoint::iterator begin, tPoint::iterator end)
- virtual bool [checkConstraints](#) (tPoint::iterator begin, tPoint::iterator end)

Public Attributes

- [spikeDensityFunction](#) * mpSDF
- double mShapeF
- double mScaleF
- double mShapeG
- double mScaleG

4.7.1 Detailed Description

uses simplex minimiser to find the best prior hyperparameters for the beta prior of the firing rate within the bins. TODO: can get stuck at constraints!

4.7.2 Member Function Documentation

4.7.2.1 virtual double findPriorExp::evaluate (tPoint::iterator *begin*, tPoint::iterator *end*)
 [inline, virtual]

Parameters:

- begin*,: iterator to first component of evaluation point
- begin*,: iterator past last component of evaluation point

Returns:

value of the function to be minimised

Reimplemented from [simplexMinimiser](#).

4.7.2.2 virtual bool findPriorExp::checkConstraints (tPoint::iterator *begin*, tPoint::iterator *end*)
 [inline, virtual]

Returns:

true if supplied point fulfills constraints

Reimplemented from [simplexMinimiser](#).

The documentation for this class was generated from the following file:

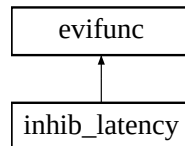
- binsdfc.cpp

4.8 inhib_latency Class Reference

probability that inhibitory [latency](#) is at [inhib_latency::mT](#), i.e. that the firing probability was above [inhib_latency::mProb](#) (this is the "inhibitory signal separation level") prior to [inhib_latency::mT](#), and is below [latency::mProb](#) for at least 1 bin after [inhib_latency::mT](#).

```
#include <evidencecomputer.h>
```

Inheritance diagram for `inhib_latency`:



Public Member Functions

- `double operator() (int &lb, int &ub, int &binno, pair< int, int > &numpts, double &evcont)`

4.8.1 Detailed Description

probability that inhibitory [latency](#) is at [inhib_latency::mT](#), i.e. that the firing probability was above [inhib_latency::mProb](#) (this is the "inhibitory signal separation level") prior to [inhib_latency::mT](#), and is below [latency::mProb](#) for at least 1 bin after [inhib_latency::mT](#).

This statement can only be true if a bin starts at [inhib_latency::mT](#). Moreover, this statement can only be true for at most 1 time index. Data in the interval ([inhib_latency::mTex1](#),[inhib_latency::mTex2](#)] are ignored.

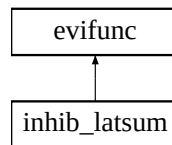
The documentation for this class was generated from the following file:

- `evidencecomputer.h`

4.9 `inhib_latsum` Class Reference

```
#include <evidencecomputer.h>
```

Inheritance diagram for `inhib_latsum`:



Public Member Functions

- `double operator() (int &lb, int &ub, int &binno, pair< int, int > &numpts, double &evcont)`

4.9.1 Detailed Description

functor for fast(er) summation of [inhib_latency](#) across all time indexes of interest, to determine the probability of the presence of an inhibitory response.

Same idea as [latsum](#) vs. [latency](#).

The documentation for this class was generated from the following file:

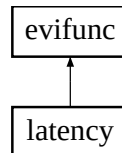
- `evidencecomputer.h`

4.10 latency Class Reference

probability that `latency` is at `latency::mT`, i.e. that the firing probability was below `latency::mProb` (this is the "signal separation level" or noise level) prior to `latency::mT`, and is above `latency::mProb` for at least 1 bin after `latency::mT`.

```
#include <evidencecomputer.h>
```

Inheritance diagram for `latency::`:



Public Member Functions

- double `operator()` (const int &lb, const int &ub, const int &binno, const pair< int, int > &numpts, const double &evcont)

4.10.1 Detailed Description

probability that `latency` is at `latency::mT`, i.e. that the firing probability was below `latency::mProb` (this is the "signal separation level" or noise level) prior to `latency::mT`, and is above `latency::mProb` for at least 1 bin after `latency::mT`.

This statement can only be true if a bin starts at `latency::mT`. Moreover, this statement can only be true for at most 1 time index.

4.10.2 Member Function Documentation

4.10.2.1 double `latency::operator()` (const int &lb, const int &ub, const int &binno, const pair< int, int > &numpts, const double &evcont) [inline]

You need to set `latency::mT`, `latency::mProb` and the beta prior exponents.

Reimplemented from `evifunc`.

The documentation for this class was generated from the following file:

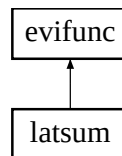
- `evidencecomputer.h`

4.11 latsum Class Reference

to compute the probability that there is a signal somewhere in the spiketrain, the averages of the latecy functor need to be summed over all time indexes in the interval of interest.

```
#include <evidencecomputer.h>
```

Inheritance diagram for latsum::



Public Member Functions

- double [operator\(\)](#) (int &lb, int &ub, int &binno, pair< int, int > &numpts, double &evcont)

4.11.1 Detailed Description

to compute the probability that there is a signal somewhere in the spiketrain, the averages of the latecy functor need to be summed over all time indexes in the interval of interest.

This can potentially take long. To speed things up, we can compute this sum a priori given that the [latency](#) is in bin [latsum::mBin](#), and then sum over all bins (which will only require the main iteration to be run M times, instead of T times).

4.11.2 Member Function Documentation

4.11.2.1 double [latsum::operator\(\)](#) (int & *lb*, int & *ub*, int & *binno*, pair< int, int > & *numpts*, double & *evcont*) [inline]

You need to set [latency::mProb](#) and the beta prior exponents.

The documentation for this class was generated from the following file:

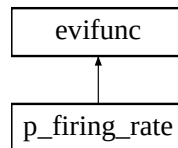
- evidencecomputer.h

4.12 p_firing_rate Class Reference

functor for the sum of [p_firing_rate_and_latency](#) across all time indexes in the interval of interest.

```
#include <evidencecomputer.h>
```

Inheritance diagram for p_firing_rate::



Public Member Functions

- void [init](#) ()
- double [operator](#)() (const int &lb, const int &ub, const int &binno, const pair< int, int > &numpts, const double &evcont)

Public Attributes

- double **mIP1m1**
- double **mIP0m1**
- int **mTm1**

4.12.1 Detailed Description

functor for the sum of [p_firing_rate_and_latency](#) across all time indexes in the interval of interest.

Instead of having to run the central iteration T times (once for each time index), we can carry out the sum over the time indexes prior to running the iteration, if we fix the bin number instead. That means that the central iteration has to be run only M times, rather than T. This yields the probability density of (the firing rate and the firing rate is above [p_firing_rate::mProb](#)).

4.12.2 Member Function Documentation

4.12.2.1 void p_firing_rate::init () [inline]

Possibly necessary initialisations. Call before starting to average.

Reimplemented from [evifunc](#).

4.12.2.2 double p_firing_rate::operator() (const int &lb, const int &ub, const int &binno, const pair< int, int > &numpts, const double &evcont) [inline]

before averaging, set [p_firing_rate::mProb](#), [p_firing_rate::mLP](#), [p_firing_rate::mL1mP](#) and [p_firing_rate::mIP1](#), [p_firing_rate::mIP0](#)

Reimplemented from [evifunc](#).

The documentation for this class was generated from the following file:

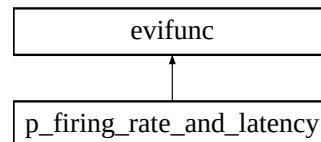
- `evidencecomputer.h`

4.13 p_firing_rate_and_latency Class Reference

functor for probability density that (firing probability==log([p_firing_rate_and_latency::mProb](#))==p_firing_rate_and_latency::mLP at [p_firing_rate_and_latency::mT](#) and firing probability prior to that was below [p_firing_rate_and_latency::mProb](#).

```
#include <evidencecomputer.h>
```

Inheritance diagram for [p_firing_rate_and_latency](#):



Public Member Functions

- void [init](#) ()
- double [operator](#)() (const int &lb, const int &ub, const int &binno, const pair< int, int > &numpts, const double &evcont)

Public Attributes

- double [mIP1m1](#)
- double [mIP0m1](#)
- int [mTm1](#)

4.13.1 Detailed Description

functor for probability density that (firing probability==log([p_firing_rate_and_latency::mProb](#))==p_firing_rate_and_latency::mLP at [p_firing_rate_and_latency::mT](#) and firing probability prior to that was below [p_firing_rate_and_latency::mProb](#).

This can only be sensibly evaluated if a bin starts at [p_firing_rate_and_latency::mT](#).

4.13.2 Member Function Documentation

4.13.2.1 void [p_firing_rate_and_latency::init](#) () [inline]

Possibly necessary initialisations. Call before starting to average.

Reimplemented from [evifunc](#).

4.13.2.2 double [p_firing_rate_and_latency::operator](#)() (const int &lb, const int &ub, const int &binno, const pair< int, int > &numpts, const double &evcont) [inline]

before averaging, set [p_firing_rate_and_latency::mT](#), [p_firing_rate_and_latency::mProb](#), [p_firing_rate_and_latency::mLP](#), [p_firing_rate_and_latency::mL1mP](#) and [p_firing_rate_and_latency::mIP1](#), [p_firing_rate_and_latency::mIP0](#)

Reimplemented from [evifunc](#).

The documentation for this class was generated from the following file:

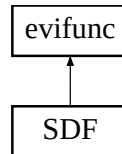
- `evidencecomputer.h`

4.14 SDF Class Reference

functor for spike density function.

```
#include <evidencecomputer.h>
```

Inheritance diagram for SDF::



Public Member Functions

- double [operator\(\)](#) (const int &lb, const int &ub, const int &binno, const pair< int, int > &numpts, const double &evcont)

4.14.1 Detailed Description

functor for spike density function.

4.14.2 Member Function Documentation

4.14.2.1 double SDF::operator() (const int & *lb*, const int & *ub*, const int & *binno*, const pair< int, int > & *numpts*, const double & *evcont*) [inline]

before starting the iteration, set [SDF::mT](#) to the time index at which you want to compute the expected PSTH/SDF, and [SDF::mIP0](#), SDF::mIP1 to the exponents of the beta prior.

Reimplemented from [evifunc](#).

The documentation for this class was generated from the following file:

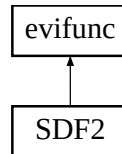
- evidencecomputer.h

4.15 SDF2 Class Reference

functor for spike density function² (to compute variance of [SDF](#))

```
#include <evidencecomputer.h>
```

Inheritance diagram for SDF2::



Public Member Functions

- double [operator\(\)](#) (const int &lb, const int &ub, const int &binno, const pair< int, int > &numpts, const double &evcont)

4.15.1 Detailed Description

functor for spike density function² (to compute variance of [SDF](#))

4.15.2 Member Function Documentation

4.15.2.1 double SDF2::operator() (const int &lb, const int &ub, const int &binno, const pair< int, int > &numpts, const double &evcont) [inline]

before starting the iteration, set [SDF::mT](#) to the time index at which you want to compute the expected PSTH/SDF², and [SDF2::mIP0](#), [SDF2::mIP1](#) to the exponents of the beta prior.

Reimplemented from [evifunc](#).

The documentation for this class was generated from the following file:

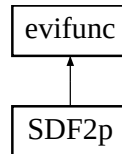
- evidencecomputer.h

4.16 SDF2p Class Reference

functor for spike density function² (to compute variance of [SDF](#)) with variable upper bound on pfire.

```
#include <evidencecomputer.h>
```

Inheritance diagram for SDF2p::



Public Member Functions

- double [operator\(\)](#) (const int &lb, const int &ub, const int &binno, const pair< int, int > &numpts, const double &evcont)

4.16.1 Detailed Description

functor for spike density function² (to compute variance of [SDF](#)) with variable upper bound on pfire.

Only use if [SDF2p::mPUB](#)<1, else you will waste time.

4.16.2 Member Function Documentation

4.16.2.1 double SDF2p::operator() (const int &lb, const int &ub, const int &binno, const pair< int, int > &numpts, const double &evcont) [inline]

before starting the iteration, set [SDF2p::mT](#) to the time index at which you want to compute the expected PSTH/SDF, [SDF2p::mPUB](#) to the upper bound on the firing prob , and [SDF2p::mIP0](#), [SDF2p::mIP1](#) to the exponents of the beta prior.

Reimplemented from [evifunc](#).

The documentation for this class was generated from the following file:

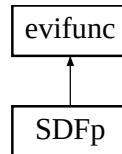
- evidencecomputer.h

4.17 SDFp Class Reference

functor for spike density function with variable upper bound on pfire.

```
#include <evidencecomputer.h>
```

Inheritance diagram for SDFp::



Public Member Functions

- double [operator\(\)](#) (const int &lb, const int &ub, const int &binno, const pair< int, int > &numpts, const double &evcont)

4.17.1 Detailed Description

functor for spike density function with variable upper bound on pfire.

Only use if [SDFp::mPUB](#)<1, else you will waste time.

4.17.2 Member Function Documentation

4.17.2.1 double SDFp::operator() (const int & lb, const int & ub, const int & binno, const pair< int, int > & numpts, const double & evcont) [inline]

before starting the iteration, set [SDFp::mT](#) to the time index at which you want to compute the expected PSTH/SDF, and [SDFp::mPUB](#) to the upper bound on the firing prob and and [SDFp::mIP0](#), SDF::mIP1 to the exponents of the beta prior.

Reimplemented from [evifunc](#).

The documentation for this class was generated from the following file:

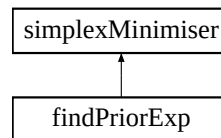
- evidencecomputer.h

4.18 simplexMinimiser Class Reference

Perform minimum search in N-dim by simplex method (see Numerical Recipes, chapter 10).

```
#include <simplexminimiser.h>
```

Inheritance diagram for simplexMinimiser::



Public Member Functions

- bool [init](#) (int dim=0)
- bool [init](#) (tPoint::iterator begin, tPoint::iterator end)
- virtual double [evaluate](#) (tPoint::iterator begin, tPoint::iterator end)
- virtual bool [checkConstraints](#) (tPoint::iterator begin, tPoint::iterator end)
- tPoint [findMinimum](#) (double tol=1e-7, int maxsteps=1000)
- bool [hasConverged](#) ()
- int [getNumSteps](#) ()
- void [test](#) ()
- void [printSimplex](#) ()

Protected Member Functions

- void [findHiLo](#) ()
- void [extrapolate](#) (double factor=-1)
- void [shrink](#) ()
- void [iterate](#) (int maxsteps)
- double [findMaxStretch](#) (double factor)
- void [computeTestpoint](#) (const double &factor)

Protected Attributes

- tSimplex [mSimplex](#)
- tPoint [mTestpoint](#)
- tPoint [mCenter](#)
- tPoint [mSearchDir](#)
- tSimplex::iterator [mHigh](#)
- tSimplex::iterator [m2ndHigh](#)
- tSimplex::iterator [mLow](#)
- double [mTol](#)
- int [mSteps](#)

4.18.1 Detailed Description

Perform minimum search in N-dim by simplex method (see Numerical Recipes, chapter 10).

Author:

Dominik Endres <dme2@st-andrews.ac.uk>

4.18.2 Member Function Documentation

4.18.2.1 `bool simplexMinimiser::init (int dim = 0)`

initialize the simplex to origin and unit vectors along the axes. If called with `dim==0`: restart simplex at current `mTestPoint` (==result from previous call to [simplexMinimiser::findMinimum\(\)](#))

Parameters:

dim,: dimensionality of space in which the search is done

Returns:

false: the start point was not valid (according to [checkConstraints\(\)](#))

4.18.2.2 `bool simplexMinimiser::init (tPoint::iterator begin, tPoint::iterator end)`

init the simplex.

Parameters:

begin,: iterator to first component of start point

end,: iterator past last component of start point

Returns:

: false, the point was not valid (according to [checkConstraints\(\)](#))

4.18.2.3 `double simplexMinimiser::evaluate (tPoint::iterator begin, tPoint::iterator end)` [virtual]

Parameters:

begin,: iterator to first component of evaluation point

begin,: iterator past last component of evaluation point

Returns:

value of the function to be minimised

Reimplemented in [findPriorExp](#).

4.18.2.4 `bool simplexMinimiser::checkConstraints (tPoint::iterator begin, tPoint::iterator end)` [virtual]

Returns:

true if supplied point fulfills constraints

Reimplemented in [findPriorExp](#).

4.18.2.5 `tPoint simplexMinimiser::findMinimum (double tol = 1e-7, int maxsteps = 1000)`

find the minimum.

Parameters:

tol,: tolerance of simplex.

maxsteps,: max number of minimisation steps

4.18.2.6 `bool simplexMinimiser::hasConverged ()`

Returns:

true if converged

4.18.2.7 `int simplexMinimiser::getNumSteps ()` [inline]

Returns:

number of steps

4.18.2.8 `void simplexMinimiser::test ()`

call to test this object

4.18.2.9 `void simplexMinimiser::findHiLo ()` [protected]

find highest, 2nd highest and lowest point, store in mHigh,m2ndHigh,mLow.

4.18.2.10 `void simplexMinimiser::extrapolate (double factor = -1)` [protected]

extrapolate high-point through the hypersurface spanned by the rest of the simplex. Store resulting point in mTestpoint. If factor==1, simplex remains unchanged (i.e. mTestpoint==mHigh)

Parameters:

factor,: extrapolation factor

4.18.2.11 `void simplexMinimiser::shrink ()` [protected]

shrink simplex around mLow by factor 0.5

4.18.2.12 void simplexMinimiser::iterate (int *maxsteps*) [protected]

simplex iteration until function does not decrease any more.

Parameters:

maxsteps,: maximal number of minimisationb steps

4.18.2.13 double simplexMinimiser::findMaxStretch (double *factor*) [protected]**Returns:**

x such that $|x| \leq |factor|$, $|x|$ maximal and $mCenter + x * mSearchDir$ fulfills constraints. set mTestpoint to this point

4.18.2.14 void simplexMinimiser::computeTestpoint (const double &*factor*) [protected]

compute testpoint by going factor times mSearchDir past mCenter

4.18.3 Member Data Documentation**4.18.3.1 tSimplex simplexMinimiser::mSimplex** [protected]

the simplex, a vector of double vectors.

4.18.3.2 tPoint simplexMinimiser::mTestpoint [protected]

test point, center of simplex w/o high point and search direction

4.18.3.3 tSimplex::iterator simplexMinimiser::mHigh [protected]

pointers to highest, 2nd highest and lowest point

4.18.3.4 double simplexMinimiser::mTol [protected]

fractional tolerance for termination

4.18.3.5 int simplexMinimiser::mSteps [protected]

number of steps so far.

The documentation for this class was generated from the following files:

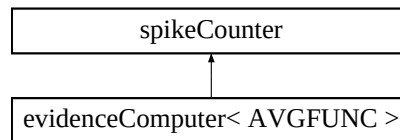
- simplexminimiser.h
- simplexminimiser.cpp

4.19 spikeCounter Class Reference

Counts spike trains and stores results.

```
#include <spikecounter.h>
```

Inheritance diagram for spikeCounter::



Public Member Functions

- void [setMaxM](#) (int m)
- void [setInterval](#) (int start, int end)
- void [setIntervalPrior](#) (double p0, double p1, double pub=1.0)
- void [clearData](#) ()
- void [addData](#) (vector< int >::iterator begin, vector< int >::iterator end)
- int [getSpikes](#) ()
- int [getGaps](#) ()
- void [test](#) (double pfire0, double pfire1, double pfire2, int numtrains)

Protected Member Functions

- bool [allocArrays](#) ()
- virtual void [precomputeSubIntervals](#) ()
- virtual bool [computePrior](#) ()

Protected Attributes

- vector< int > [mCurSp](#)

Static Protected Attributes

- static vector< vector< double > > [mIntervalEvidences](#)
- static vector< vector< pair< int, int > > > [mIntervalCounts](#)
- static vector< double > [mPriors](#)
- static double [mPrior1](#)
- static double [mPrior0](#)
- static int [mIntervStart](#)
- static int [mIntervEnd](#)
- static int [mMMax](#)
- static vector< pair< int, int > > [mSpikeTrain](#)
- static vector< vector< int > > [mAutoCorr](#)
- static bool [mbDataChanged](#)
- static double [mPUB](#)

4.19.1 Detailed Description

Counts spike trains and stores results.

A spike train is represented as a `std::vector<int>`, each `int` being the time index when a spike was observed. Most members are static, so that data and precomputed priors and interval evidences can be shared across instances of objects which inherit from this object, without having to duplicate data. [spikeCounter::mIntervalEvidences](#) are marginal probabilities of each interval under a binomial observation model with a beta prior on the success (i.e. spike) probability of the binomial. If you want to change the observation model, derive from this object.

Author:

Dominik Endres

4.19.2 Member Function Documentation

4.19.2.1 void spikeCounter::setMaxM (int *m*)

set maximal number of bin boundaries

Parameters:

m,: maximal number of bin boundaries

4.19.2.2 void spikeCounter::setInterval (int *start*, int *end*)

set interval within which [SDF](#) is computed

4.19.2.3 void spikeCounter::setIntervalPrior (double *p0*, double *p1*, double *pub* = 1.0)

prior exponents for Beta distribution $p^{p1} \cdot (1-p)^{p0}$ in every bin. Must be $> 1e-5$

Parameters:

p0,: exponent for "no spike"

p1,: exponent for "spike"

4.19.2.4 void spikeCounter::clearData ()

clears the spiketrain

4.19.2.5 void spikeCounter::addData (vector< int >::iterator *begin*, vector< int >::iterator *end*)

add a run (from `dengkeData`) to the current spiketrain

Parameters:

begin,: iterator to beginning of spike train to be added.

end,: iterator past its end.

4.19.2.6 int spikeCounter::getSpikes ()**Returns:**

total number of spikes

4.19.2.7 int spikeCounter::getGaps ()**Returns:**

total number of gaps (i.e. observed time indexes with no spikes)

4.19.2.8 void spikeCounter::test (double *pfire0*, double *pfire1*, double *pfire2*, int *numtrains*)

test code. Generate some spike trains and count

Parameters:

pfire0,: baseline firing probability
pfire1,: transient firing probability
pfire2,: sustained firing probability
numtrains,: number of spike trains to generate

4.19.2.9 bool spikeCounter::allocArrays () [protected]

allocate storage for mIntervalEvidences and mIntervalCounts

4.19.2.10 void spikeCounter::precomputeSubIntervals () [protected, virtual]

precompute counts and sub-evidences from current spiketrain.

4.19.2.11 bool spikeCounter::computePrior () [protected, virtual]

compute M-priors. Return false if impossible (i.e. mMMax>mIntervEnd-mIntervStart-1)

4.19.3 Member Data Documentation**4.19.3.1 vector< vector< double > > spikeCounter::mIntervalEvidences [static, protected]**

Evidences of sub-intervals. The return values of the function getIEC() in [Endres, Oram, Schindelin, Foldiak \(2007\): Bayesian binning beats approximate alternatives: estimating peri-stimulus time histograms](#)

4.19.3.2 vector< vector< pair< int, int > > > spikeCounter::mIntervalCounts [static, protected]

counts of spikes and gaps in the sub-intervals

4.19.3.3 `vector< double > spikeCounter::mPriors` [static, protected]

priors for a given bin number

4.19.3.4 `double spikeCounter::mPrior1` [static, protected]

priors of the beta distributions in the sub-intervals

4.19.3.5 `int spikeCounter::mIntervStart` [static, protected]

beginning and end (exclusive) of time interval within which [SDF](#) is computed

4.19.3.6 `int spikeCounter::mMMax` [static, protected]

maximal number of bin boundaries (bins-1) in the time interval

4.19.3.7 `vector< pair< int, int > > spikeCounter::mSpikeTrain` [static, protected]

the marginal spiketrain. Stores pairs of <spikes,gaps> for every time index in the interval

4.19.3.8 `vector<int> spikeCounter::mCurSp` [protected]

the current spiketrain

4.19.3.9 `vector< vector< int > > spikeCounter::mAutoCorr` [static, protected]

autocorrelation counts of the spiketrain. `mAutoCorr[i][j<i]`=single spike train autocorrelation for time indexes i,j

4.19.3.10 `bool spikeCounter::mbDataChanged` [static, protected]

true if data have been changed since last call to `precomputeSubIntervals`

4.19.3.11 `double spikeCounter::mPUB` [static, protected]

upper bound on firing prob.

The documentation for this class was generated from the following files:

- spikecounter.h
- spikecounter.cpp

4.20 spikeDensityFunction Class Reference

compute bin spike density function and related quantities, e.g. [latency](#), window end, signal probability.

```
#include <spikedensityfunction.h>
```

Public Member Functions

- void [setMaxM](#) (int mmax)
- void [setIntervalPriors](#) (double p0, double p1, double pub=1.0)
- void [setTimeInterval](#) (int start, int end)
- void [clearData](#) ()
- void [addData](#) (vector< int >::iterator begin, vector< int >::iterator end)
- double [findMInterval](#) (double pmin, int &mmin, int &mmax)
- void [setMInterval](#) (int m1, int m2)
- double [getMProb](#) (int m)
- double [getSDF](#) (int tind)
- double [getSDFStdDev](#) (int tind, double sdfval)
- double [getBBProb](#) (int m, int tind)
- double [getBBExpectation](#) (int m)
- double [getBBStdDev](#) (int m, double bbexp)
- double [getPAbNoise](#) (int tind, double noiselevel)
- double [getLatencyProb](#) (int tind, double noiselevel)
- void [getLatencyProbs](#) (int tind, double noiselevel, vector< double > &res)
- double [getBABProb](#) (int tstart, int tend, double noiselevel)
- double [getNOSIG](#) (double noiselevel)
- double [getSeparationProb](#) (double noiselevel, int tstart=INT_MIN, int tend=INT_MAX)
- list< pair< double, double > > [findBestSeparator](#) (double lbound, double ubound, double &noiselevel, double &prob, int tstart=INT_MIN, int tend=INT_MAX)
- double [getFiringRateGivenLatencyProb](#) (int tind, double pfire, double noiselevel)
- double [getFiringRateProb](#) (double pfire, double noiselevel, int tstart=INT_MIN, int tend=INT_MAX)
- double [getTotalEvidence](#) ()
- double [getIntervalEvidenceSum](#) ()
- double [getEvidence](#) (int m)
- double [getInhibitoryLatencyProb](#) (int tind, double inhibitory_noiselevel)
- double [getInhibitorySeparationProb](#) (double inhibitory_noiselevel, int tstart=INT_MIN, int tend=INT_MAX)
- list< pair< double, double > > [findBestInhibitorySeparator](#) (double lbound, double ubound, double &noiselevel, double &prob, int tstart=INT_MIN, int tend=INT_MAX)
- void [setExcludedTimeInterval](#) (int ts, int te)

Public Attributes

- int [mMaxM](#)

Protected Member Functions

- list< pair< double, double > > [goldenSectionSearch](#) (double lbound, double ubound, double &maxpos, double &max, int tstart, int tend, double(spikeDensityFunction::*maxfunc)(double, int, int))
- void [computeEvidences](#) ()
- double [getBABBinProb](#) (int bin1, int bin2, double noiselevel)

Protected Attributes

- [evidenceComputer](#)< [evifunc](#) > [mEV](#)
- [evidenceComputer](#)< [binboundary_expectation](#) > [mBBE](#)
- [evidenceComputer](#)< [binboundary_expectation2](#) > [mBBE2](#)
- [evidenceComputer](#)< [binboundary_probability](#) > [mBBP](#)
- [evidenceComputer](#)< [SDF](#) > [mSDF](#)
- [evidenceComputer](#)< [SDFp](#) > [mSDFp](#)
- [evidenceComputer](#)< [SDF2](#) > [mSDF2](#)
- [evidenceComputer](#)< [SDF2p](#) > [mSDF2p](#)
- [evidenceComputer](#)< [aboveNoise](#) > [mABNOI](#)
- [evidenceComputer](#)< [latency](#) > [mLAT](#)
- [evidenceComputer](#)< [nssns](#) > [mBAB](#)
- [evidenceComputer](#)< [nssns_bin](#) > [mBABBin](#)
- [evidenceComputer](#)< [nosig](#) > [mNOS](#)
- [evidenceComputer](#)< [latsum](#) > [mLatSum](#)
- [evidenceComputer](#)< [inhib_latsum](#) > [mInhibitoryLatSum](#)
- [evidenceComputer](#)< [inhib_latency](#) > [mInhibitoryLatency](#)
- [evidenceComputer](#)< [p_firing_rate_and_latency](#) > [mPFRAL](#)
- [evidenceComputer](#)< [p_firing_rate](#) > [mPFR](#)
- [spikeCounter](#) [mSP](#)
- int [mM1](#)
- int [mM2](#)
- double [mP0](#)
- double [mP1](#)
- double [mPUB](#)
- int [mIntervStart](#)
- int [mIntervEnd](#)
- bool [mbDataChanged](#)
- bool [mbMIntervalChanged](#)
- double [mEvSum](#)
- double [mEvIntervSum](#)
- double [mWndstartIntervSum](#)
- int [mWndstartLastTime](#)
- double [mNoiselevelLastTime](#)
- int [mTex1](#)
- int [mTex2](#)

4.20.1 Detailed Description

compute bin spike density function and related quantities, e.g. [latency](#), window end, signal probability.

Author:

Dominik Endres

4.20.2 Member Function Documentation

4.20.2.1 void spikeDensityFunction::setMaxM (int *mmax*)

set maximal number of bin boundaries

4.20.2.2 void spikeDensityFunction::setIntervalPriors (double *p0*, double *p1*, double *pub* = 1.0)

set prior exponents of the Beta distributions for each interval

4.20.2.3 void spikeDensityFunction::setTimeInterval (int *start*, int *end*)

time interval within which sdf etc. is computed

4.20.2.4 void spikeDensityFunction::clearData ()

clears all spike data and precomputed arrays

4.20.2.5 void spikeDensityFunction::addData (vector< int >::iterator *begin*, vector< int >::iterator *end*)

add a stimulus run to the data

4.20.2.6 double spikeDensityFunction::findMInterval (double *pmin*, int & *mmin*, int & *mmax*)

find an interval of bin boundary numbers such that the probability that the correct model is included is at least *pmin*.

Returns:

the actual probability

4.20.2.7 void spikeDensityFunction::setMInterval (int *m1*, int *m2*)

set interval of bin boundary numbers to be used in subsequent inferences.

4.20.2.8 double spikeDensityFunction::getMProb (int *m*)

Returns:

posterior probability of a model with *m* bin boundaries. Assumes uniform *M* prior.

4.20.2.9 double spikeDensityFunction::getSDF (int *tind*)

Returns:

expected [SDF](#) at time index *tind*

4.20.2.10 double spikeDensityFunction::getSDFStdDev (int *tind*, double *sdfval*)**Returns:**

expected [SDF](#) standard deviation.

Parameters:

tind,: time index.

sdfval,: expexted sdf at tind.

4.20.2.11 double spikeDensityFunction::getBBProb (int *m*, int *tind*)**Returns:**

probability that bin boundary *m* is at time index *tind*

4.20.2.12 double spikeDensityFunction::getBBExpectation (int *m*)**Returns:**

expectation of bin boundary *m*

4.20.2.13 double spikeDensityFunction::getBBStdDev (int *m*, double *bbexp*)**Returns:**

standard deviation of bin boundary *m*, given its expectation

4.20.2.14 double spikeDensityFunction::getPAbNoise (int *tind*, double *noiselevel*)**Returns:**

probability that signal is above *noiselevel* (==noise firing probability) at time index *tind*

4.20.2.15 double spikeDensityFunction::getLatencyProb (int *tind*, double *noiselevel*)**Returns:**

probability that [latency](#) is at time index *tind* given *noiselevel* (i.e. prob that signal is below *noiselevel* before *tind*, and above *noiselevel* for at least one bin starting at *tind*).

Parameters:

tind,: time index

noiselevel,: signal separation level, i.e. firing probability below which there's no signal (i.e. noise) and above which there is a signal.

4.20.2.16 void spikeDensityFunction::getLatencyProbs (int *tind*, double *noiselevel*, vector<double> & *res*)

compute probabilities that [latency](#) is at *tind* given *M*, for all *M* in the interval set by [getLatencyProb\(\)](#).

Parameters:

tind,: time index

noiselevel,: signal separation level

res,: vector of [latency](#) probabilities for each *M*

4.20.2.17 double spikeDensityFunction::getBABProb (int *tstart*, int *tend*, double *noiselevel*)

Returns:

probability that the signal in the spiketrain starts at *tstart* and ends at *tend* (i.e. is below the supplied noise level prior to *tstart*, above it between *tstart* and *tend*, and below after *tend*), given that there is a signal in the spiketrain

4.20.2.18 double spikeDensityFunction::getNOSIG (double *noiselevel*)

Returns:

probability that there is no signal in the spiketrain

4.20.2.19 double spikeDensityFunction::getSeparationProb (double *noiselevel*, int *tstart* = INT_MIN, int *tend* = INT_MAX)

probability that separation between below/above *noiselevel* is possible

4.20.2.20 list< pair< double, double > > spikeDensityFunction::findBestSeparator (double *lbound*, double *ubound*, double & *noiselevel*, double & *prob*, int *tstart* = INT_MIN, int *tend* = INT_MAX)

find best separation level.

Parameters:

lbound,: lower bound on search interval for separation level.

ubound. upper bound on search interval for separation level.

noiselevel,: best separation level

prob,: probability of signal at best separation level.

Returns:

list of pairs (level,prob) that have been tried by golden section search. *noiselevel* will be the best level, *prob* the probability that there's a signal at that *noiselevel*.

4.20.2.21 `double spikeDensityFunction::getFiringRateGivenLatencyProb (int tind, double pfire, double noiselevel)`

Returns:

probability that firing rate is *pfire*, given [latency](#) at *tind* and *noiselevel*

4.20.2.22 `double spikeDensityFunction::getFiringRateProb (double pfire, double noiselevel, int tstart = INT_MIN, int tend = INT_MAX)`

Returns:

probability that firing rate is *pfire*, given *noiselevel*

4.20.2.23 `double spikeDensityFunction::getTotalEvidence ()`

Returns:

total evidence $P(D)$, assuming a uniform prior over M

4.20.2.24 `double spikeDensityFunction::getIntervalEvidenceSum ()`

Returns:

$\log \sum P(D|M)$, summing over the interval set in [getLatencyProb\(\)](#).

4.20.2.25 `double spikeDensityFunction::getEvidence (int m)`

Returns:

$\log P(D|m)$

4.20.2.26 `double spikeDensityFunction::getInhibitoryLatencyProb (int tind, double inhibitory_noiselevel)`

Parameters:

tind,: time index.

inhibitory_noiselevel,: inhibitory signal separation level. Inverse of (excitatory) signal separation level.

Returns:

probability that inhibitory [latency](#) is at *tind*.

4.20.2.27 `double spikeDensityFunction::getInhibitorySeparationProb (double inhibitory_noiselevel, int tstart = INT_MIN, int tend = INT_MAX)`

Parameters:

inhibitory_noiselevel,: inhibitory signal separation level.
tstart,: beginning of tomer i

Returns:

probability that separation between above/below inhibitory noiselevel is possible

4.20.2.28 `list< pair< double, double > > spikeDensityFunction::findBestInhibitorySeparator (double lbound, double ubound, double & noiselevel, double & prob, int tstart = INT_MIN, int tend = INT_MAX)`

find best separation level between lbound and ubound. Returns list of values (level,prob) that have been tried by golden section search. noiselevel will be the best level, prob its probability

4.20.2.29 `list< pair< double, double > > spikeDensityFunction::goldenSectionSearch (double lbound, double ubound, double & maxpos, double & max, int tstart, int tend, double(spikeDensityFunction::*)(double, int, int) maxfunc)` [protected]

golden section maximum search.

Parameters:

lbound,: lower bound of search interval
ubound,: upper bound of search interval
maxpos,: position of maximum
max,: value at maximum
maxfunc,: function to be maximised.

Returns:

list of pairs of (search point, value at search point).

4.20.2.30 `void spikeDensityFunction::computeEvidences ()` [protected]

compute evidences. mbDataChanged=false if successful

4.20.3 Member Data Documentation

4.20.3.1 `int spikeDensityFunction::mMaxM`

max number of bin boundaries

4.20.3.2 `evidenceComputer<evifunc> spikeDensityFunction::mEV` [protected]

template instances of the central iteration with different averaging functors. See documentation of [evidenceComputer](#)

4.20.3.3 int spikeDensityFunction::mM1 [protected]

smallest/largest number of bin boundaries used in inference

4.20.3.4 double spikeDensityFunction::mP0 [protected]

prior exponents of Beta distribution for each interval, and upper bound on firing prob

4.20.3.5 int spikeDensityFunction::mIntervStart [protected]

time interval within which [SDF](#) etc. is computed

4.20.3.6 bool spikeDensityFunction::mbDataChanged [protected]

true if data changed since last evidence computation

4.20.3.7 bool spikeDensityFunction::mbMIntervalChanged [protected]

true if M-interval (i.e. mM1 or mM2) changed since last computation of evidence sums

4.20.3.8 double spikeDensityFunction::mEvSum [protected]

total evidence sum

4.20.3.9 double spikeDensityFunction::mEvIntervSum [protected]

evidence sum for $mM1 \leq m \leq mM2$. Same for p_wndstart functor

4.20.3.10 int spikeDensityFunction::mWndstartLastTime [protected]

mWndstartIntervSum needs only be recomputed if one of those differs from the last call to `getFiringRateProb`

4.20.3.11 int spikeDensityFunction::mTex1 [protected]

excluded time interval for [latency](#) calculations (e.g. because it contains stimulation artefacts)

The documentation for this class was generated from the following files:

- `spikedensityfunction.h`
- `spikedensityfunction.cpp`

Index

- aboveNoise, [9](#)
 - operator(), [9](#)
- addData
 - spikeCounter, [39](#)
 - spikeDensityFunction, [44](#)
- allocArrays
 - spikeCounter, [40](#)
- binboundary_expectation, [11](#)
 - operator(), [11](#)
- binboundary_expectation2, [12](#)
 - operator(), [12](#)
- binboundary_probability, [13](#)
 - operator(), [13](#)
- checkConstraints
 - findPriorExp, [20](#)
 - simplexMinimiser, [35](#)
- clearData
 - spikeCounter, [39](#)
 - spikeDensityFunction, [44](#)
- compute
 - evidenceComputer, [15](#)
- computeEvidences
 - evidenceComputer, [15](#)
 - spikeDensityFunction, [48](#)
- computePrior
 - spikeCounter, [40](#)
- computeTestpoint
 - simplexMinimiser, [37](#)
- evaluate
 - findPriorExp, [20](#)
 - simplexMinimiser, [35](#)
- evidenceComputer, [14](#)
 - compute, [15](#)
 - computeEvidences, [15](#)
 - getEvidence, [15](#)
 - getEvidenceSum, [15](#)
 - mAF, [15](#)
 - mEvidences, [15](#)
 - mSubEvidences, [16](#)
 - setFunctorParams, [15](#)
 - test, [15](#)
- evifunc, [17](#)
 - init, [18](#)
 - mBin, [18](#)
 - mIPO, [19](#)
 - mProb, [18](#)
 - mPUB, [19](#)
 - mT, [18](#)
 - operator(), [18](#)
- extrapolate
 - simplexMinimiser, [36](#)
- findBestInhibitorySeparator
 - spikeDensityFunction, [48](#)
- findBestSeparator
 - spikeDensityFunction, [46](#)
- findHiLo
 - simplexMinimiser, [36](#)
- findMaxStretch
 - simplexMinimiser, [37](#)
- findMinimum
 - simplexMinimiser, [36](#)
- findMInterval
 - spikeDensityFunction, [44](#)
- findPriorExp, [20](#)
 - checkConstraints, [20](#)
 - evaluate, [20](#)
- getBABProb
 - spikeDensityFunction, [46](#)
- getBBExpectation
 - spikeDensityFunction, [45](#)
- getBBProb
 - spikeDensityFunction, [45](#)
- getBBStdDev
 - spikeDensityFunction, [45](#)
- getEvidence
 - evidenceComputer, [15](#)
 - spikeDensityFunction, [47](#)
- getEvidenceSum
 - evidenceComputer, [15](#)
- getFiringRateGivenLatencyProb
 - spikeDensityFunction, [46](#)
- getFiringRateProb
 - spikeDensityFunction, [47](#)
- getGaps
 - spikeCounter, [40](#)

- getInhibitoryLatencyProb
 - spikeDensityFunction, 47
- getInhibitorySeparationProb
 - spikeDensityFunction, 47
- getIntervalEvidenceSum
 - spikeDensityFunction, 47
- getLatencyProb
 - spikeDensityFunction, 45
- getLatencyProbs
 - spikeDensityFunction, 45
- getMProb
 - spikeDensityFunction, 44
- getNOSIG
 - spikeDensityFunction, 46
- getNumSteps
 - simplexMinimiser, 36
- getPAbNoise
 - spikeDensityFunction, 45
- getSDF
 - spikeDensityFunction, 44
- getSDFStdDev
 - spikeDensityFunction, 44
- getSeparationProb
 - spikeDensityFunction, 46
- getSpikes
 - spikeCounter, 39
- getTotalEvidence
 - spikeDensityFunction, 47
- goldenSectionSearch
 - spikeDensityFunction, 48
- hasConverged
 - simplexMinimiser, 36
- inhib_latency, 22
- inhib_latsum, 23
- init
 - evifunc, 18
 - p_firing_rate, 26
 - p_firing_rate_and_latency, 28
 - simplexMinimiser, 35
- iterate
 - simplexMinimiser, 36
- latency, 24
 - operator(), 24
- latsum, 25
 - operator(), 25
- mAF
 - evidenceComputer, 15
- mAutoCorr
 - spikeCounter, 41
- mbDataChanged
 - spikeCounter, 41
 - spikeDensityFunction, 49
- mBin
 - evifunc, 18
- mbMIntervalChanged
 - spikeDensityFunction, 49
- mCurSp
 - spikeCounter, 41
- mEV
 - spikeDensityFunction, 48
- mEvidences
 - evidenceComputer, 15
- mEvIntervSum
 - spikeDensityFunction, 49
- mEvSum
 - spikeDensityFunction, 49
- mHigh
 - simplexMinimiser, 37
- mIntervalCounts
 - spikeCounter, 40
- mIntervalEvidences
 - spikeCounter, 40
- mIntervStart
 - spikeCounter, 41
 - spikeDensityFunction, 49
- mIP0
 - evifunc, 19
- mM1
 - spikeDensityFunction, 48
- mMaxM
 - spikeDensityFunction, 48
- mMMax
 - spikeCounter, 41
- mP0
 - spikeDensityFunction, 49
- mPrior1
 - spikeCounter, 41
- mPriors
 - spikeCounter, 40
- mProb
 - evifunc, 18
- mPUB
 - evifunc, 19
 - spikeCounter, 41
- mSimplex
 - simplexMinimiser, 37
- mSpikeTrain
 - spikeCounter, 41
- mSteps
 - simplexMinimiser, 37
- mSubEvidences
 - evidenceComputer, 16
- mT
 - evifunc, 18

- mTestpoint
 - simplexMinimiser, 37
- mTex1
 - spikeDensityFunction, 49
- mTol
 - simplexMinimiser, 37
- mWndstartLastTime
 - spikeDensityFunction, 49
- operator()
 - aboveNoise, 9
 - binboundary_expectation, 11
 - binboundary_expectation2, 12
 - binboundary_probability, 13
 - evifunc, 18
 - latency, 24
 - latsum, 25
 - p_firing_rate, 26
 - p_firing_rate_and_latency, 28
 - SDF, 30
 - SDF2, 31
 - SDF2p, 32
 - SDFp, 33
- p_firing_rate, 26
 - init, 26
 - operator(), 26
- p_firing_rate_and_latency, 28
 - init, 28
 - operator(), 28
- precomputeSubIntervals
 - spikeCounter, 40
- SDF, 30
 - operator(), 30
- SDF2, 31
 - operator(), 31
- SDF2p, 32
 - operator(), 32
- SDFp, 33
 - operator(), 33
- setFunctorParams
 - evidenceComputer, 15
- setInterval
 - spikeCounter, 39
- setIntervalPrior
 - spikeCounter, 39
- setIntervalPriors
 - spikeDensityFunction, 44
- setMaxM
 - spikeCounter, 39
 - spikeDensityFunction, 44
- setMInterval
 - spikeDensityFunction, 44
- setTimeInterval
 - spikeDensityFunction, 44
- shrink
 - simplexMinimiser, 36
- simplexMinimiser, 34
 - checkConstraints, 35
 - computeTestpoint, 37
 - evaluate, 35
 - extrapolate, 36
 - findHiLo, 36
 - findMaxStretch, 37
 - findMinimum, 36
 - getNumSteps, 36
 - hasConverged, 36
 - init, 35
 - iterate, 36
 - mHigh, 37
 - mSimplex, 37
 - mSteps, 37
 - mTestpoint, 37
 - mTol, 37
 - shrink, 36
 - test, 36
- spikeCounter, 38
 - addData, 39
 - allocArrays, 40
 - clearData, 39
 - computePrior, 40
 - getGaps, 40
 - getSpikes, 39
 - mAutoCorr, 41
 - mbDataChanged, 41
 - mCurSp, 41
 - mIntervalCounts, 40
 - mIntervalEvidences, 40
 - mIntervStart, 41
 - mMMax, 41
 - mPrior1, 41
 - mPriors, 40
 - mPUB, 41
 - mSpikeTrain, 41
 - precomputeSubIntervals, 40
 - setInterval, 39
 - setIntervalPrior, 39
 - setMaxM, 39
 - test, 40
- spikeDensityFunction, 42
 - addData, 44
 - clearData, 44
 - computeEvidences, 48
 - findBestInhibitorySeparator, 48
 - findBestSeparator, 46
 - findMInterval, 44
 - getBABProb, 46

- getBBExpectation, [45](#)
- getBBProb, [45](#)
- getBBStdDev, [45](#)
- getEvidence, [47](#)
- getFiringRateGivenLatencyProb, [46](#)
- getFiringRateProb, [47](#)
- getInhibitoryLatencyProb, [47](#)
- getInhibitorySeparationProb, [47](#)
- getIntervalEvidenceSum, [47](#)
- getLatencyProb, [45](#)
- getLatencyProbs, [45](#)
- getMProb, [44](#)
- getNOSIG, [46](#)
- getPAbNoise, [45](#)
- getSDF, [44](#)
- getSDFStdDev, [44](#)
- getSeparationProb, [46](#)
- getTotalEvidence, [47](#)
- goldenSectionSearch, [48](#)
- mbDataChanged, [49](#)
- mbMIntervalChanged, [49](#)
- mEV, [48](#)
- mEvIntervSum, [49](#)
- mEvSum, [49](#)
- mIntervStart, [49](#)
- mM1, [48](#)
- mMaxM, [48](#)
- mP0, [49](#)
- mTex1, [49](#)
- mWndstartLastTime, [49](#)
- setIntervalPriors, [44](#)
- setMaxM, [44](#)
- setMInterval, [44](#)
- setTimeInterval, [44](#)

test

- evidenceComputer, [15](#)
- simplexMinimiser, [36](#)
- spikeCounter, [40](#)